BurstSort

Solução eficiente para ordenamento de strings com Access Trie

Cache-Conscious Sorting of Large Sets of Strings with Dynamic Tries

RANJAN SINHA and JUSTIN ZOBEL
RMIT University

Ongoing changes in computer architecture are affecting the efficiency of string-sorting algorithms. The size of main memory in typical computers continues to grow but memory accesses require increasing numbers of instruction cycles, which is a problem for the most efficient of the existing string-sorting algorithms as they do not utilize cache well for large data sets. We propose a new sorting algorithm for strings, burstsort, based on dynamic construction of a compact trie in which strings are kept in buckets. It is simple, fast, and efficient. We experimentally explore key implementation options and compare burstsort to existing string-sorting algorithms on large and small sets of strings with a range of characteristics. These experiments show that, for large sets of strings, burstsort is almost twice as fast as any previous algorithm, primarily due to a lower rate of cache miss.

Categories and Subject Descriptors: F.2.2 [**Analysis of Algorithms**]: Sorting; E.5 [**Files**]: Sorting; E.1 [**Data Structures**]: Trees; B.3.2 [**Memory Structures**]: Cache Memories; D.1.0 [**Programming Techniques**]: General

https://doi.org/10.1145/1005813.1041517

O Problema: Latência de Memória

1. A Desigualdade do Hardware

- **Velocidade vs. Capacidade:** Processadores operam em GHz , mas a memória principal (DRAM) é lenta.
- **O Refúgio do Cache:** Pequenas memórias ultra-rápidas (L1/L2) que o algoritmo deve priorizar.
- **O Abismo:** Um acesso à RAM pode custar até **200 ciclos** de processamento desperdiçado.

2. Por que algoritmos clássicos falham?

- **O custo do Ponteiro:** Acessar uma string via ponteiro causa **2 cache misses** (um para o endereço, outro para o dado).
- **Radixsort (Breadth-first):** Analisa um caractere por vez de todas as strings. Força o sistema a buscar a mesma string na RAM repetidamente.
- **Quicksort:** À medida que o conjunto cresce, as comparações tornam-se aleatórias, "saltando" pela memória e esvaziando o cache.

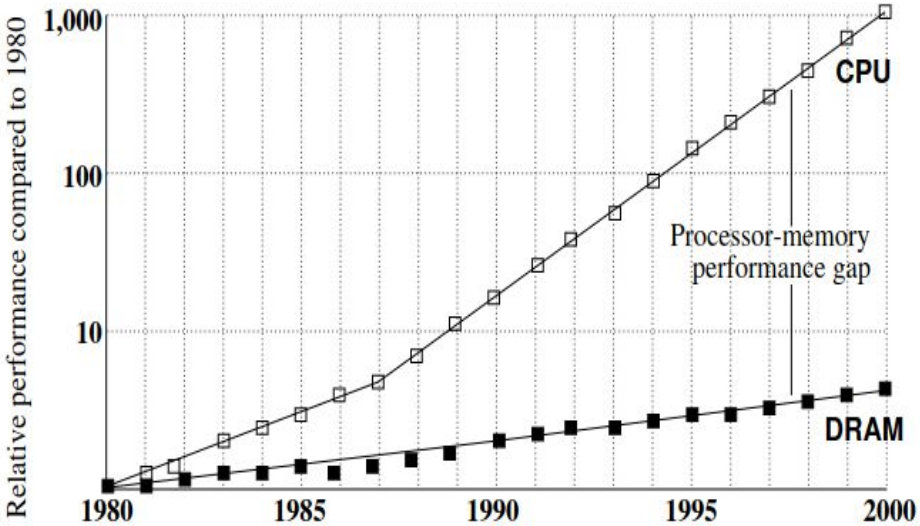


Figure 1.1: Processor-memory performance gap

https://doi.org/10.13140/RG.2.2.14587.75045

Componentes Estruturais

Access Trie (O Guia):

- Nós contendo arrays de ponteiros (representando o alfabeto).
- Direciona a string até o balde correto com base nos seus caracteres iniciais.

Buckets (Os Baldes):

- Estruturas pequenas e eficientes que armazenam strings com o mesmo prefixo.
- Localizados nas "folhas" da Trie.

Empty-string Pointers:

- Ponteiros especiais para strings que terminam exatamente em um nó (ex: a palavra "casa" dentro de uma trie que já processou "casa-mento").

“bat”, “barn”, “bark”, “by”, “by”, “by”, “by”, “byte”, “bytes”, “wane”, “way”, e “west”

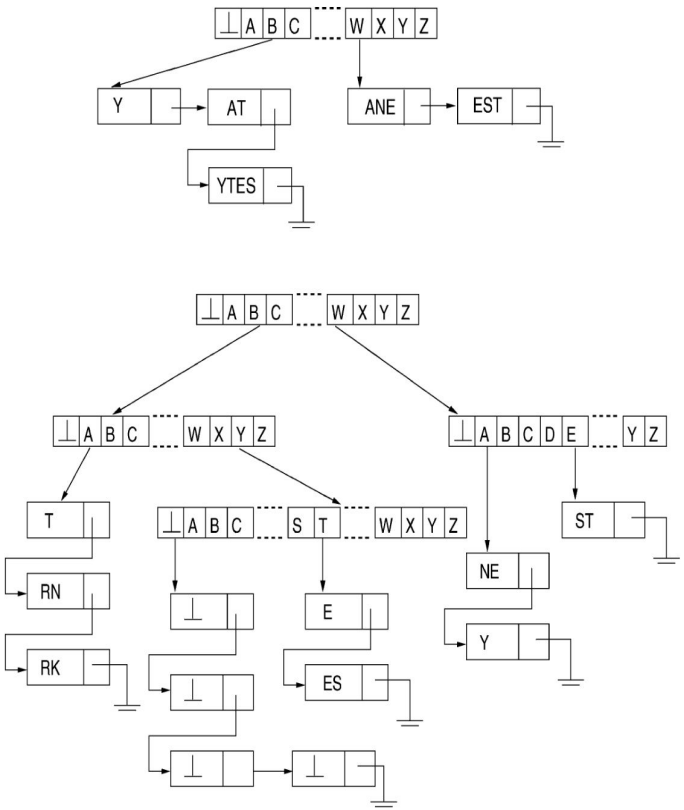


Fig. 1. Burst trie using lists, with four trie nodes, five buckets, and twelve strings. Upper: before any bursts. Lower: after all bursts.

https://doi.org/10.1145/1005813.1041517

Operações Relevantes

Inserção:

- As strings são inseridas uma por uma.
- **Vantagem:** O processador consome a string inteira de uma vez, mantendo-a no cache (*cache-friendly*).

Explosão (Bursting):

- Quando um balde ultrapassa o limite **L**, ele "explode".
- O balde é substituído por um novo nó da Trie e as strings são redistribuídas em novos sub-baldes.

Travessia e Ordenação Final:

- Percorre a Trie (Depth-first).
- Baldes pequenos são ordenados individualmente usando **Multikey Quicksort**.
- Como os baldes são pequenos, a ordenação ocorre inteiramente dentro do Cache.

Parâmetros de Otimização

Recurso	Escolha de Implementação	Justificativa
Representação do Bucket	Lista ligada de arrays	Combina a estabilidade e a inserção barata das listas com a eficiência de cache dos arrays.
Estratégia de Crescimento	Exponencial	Os buckets começam pequenos (ex: 16 ponteiros) e crescem por um fator (ex: 8) até atingirem o limite de explosão, minimizando a fragmentação de memória.
Limite de Explosão (L)	8.192 strings	Equilibra o tamanho da trie com o custo de ordenar os buckets individualmente.
Tamanho do Nó Raiz	2^16 (16-bit)	Permitir que o nó raiz consuma dois caracteres de uma vez reduz o número de nós alocados dinamicamente e economiza operações.
Ordenação do Bucket	Multikey quicksort	Provou-se ser a rotina mais eficiente para ordenar os pequenos conjuntos de strings encontrados dentro dos buckets.

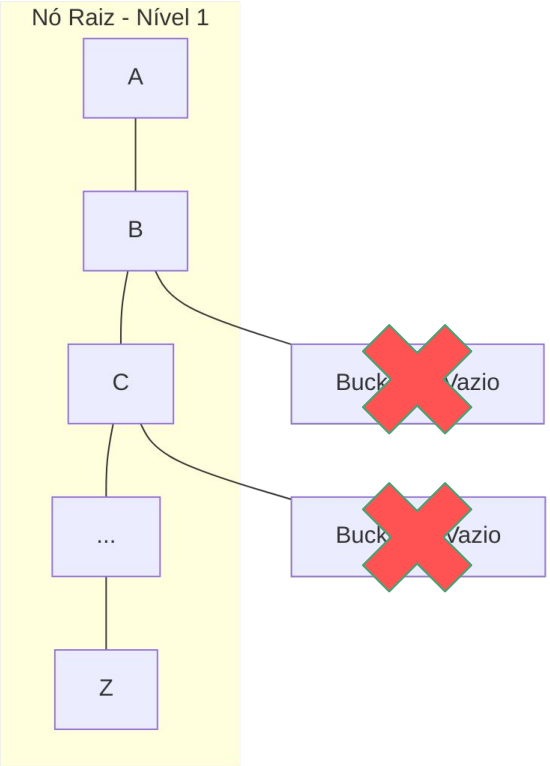
Exemplo

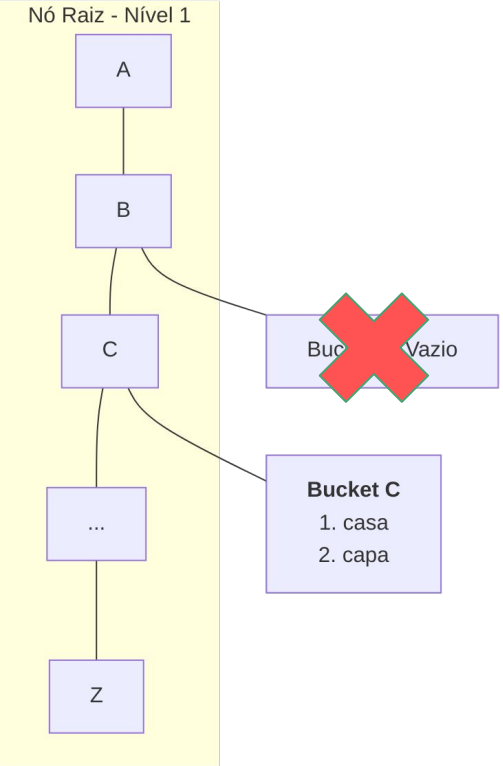
Vamos usar:

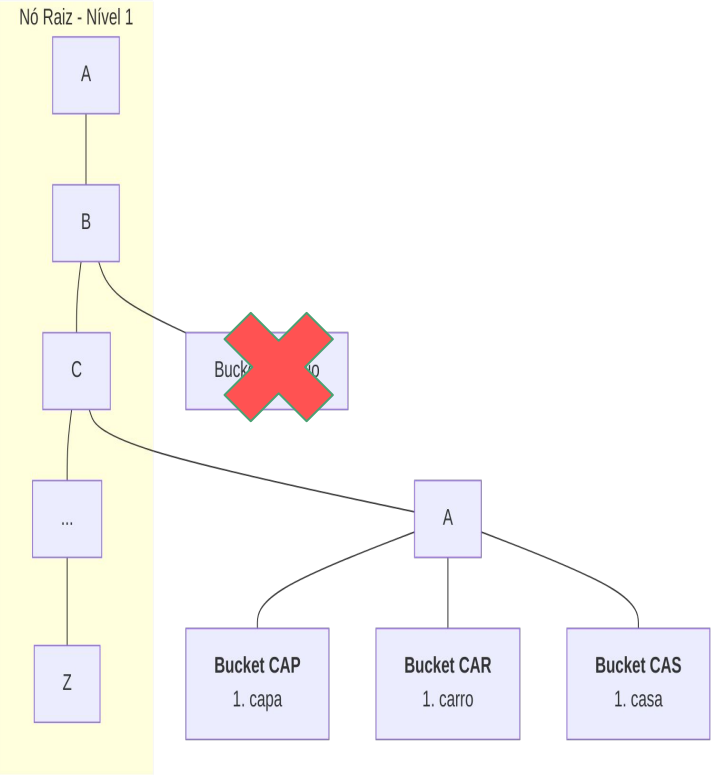
- **Dados:** `["casa", "capa", "carro", "casa", "bola", "bota"]`
- **Limite de Explosão (L):** 2 strings.
- **Nó Raiz:** Simplificado para 1 caractere.

```
["casa", "capa", "carro", "casa", "bola", "bota"]
```

Exemplo

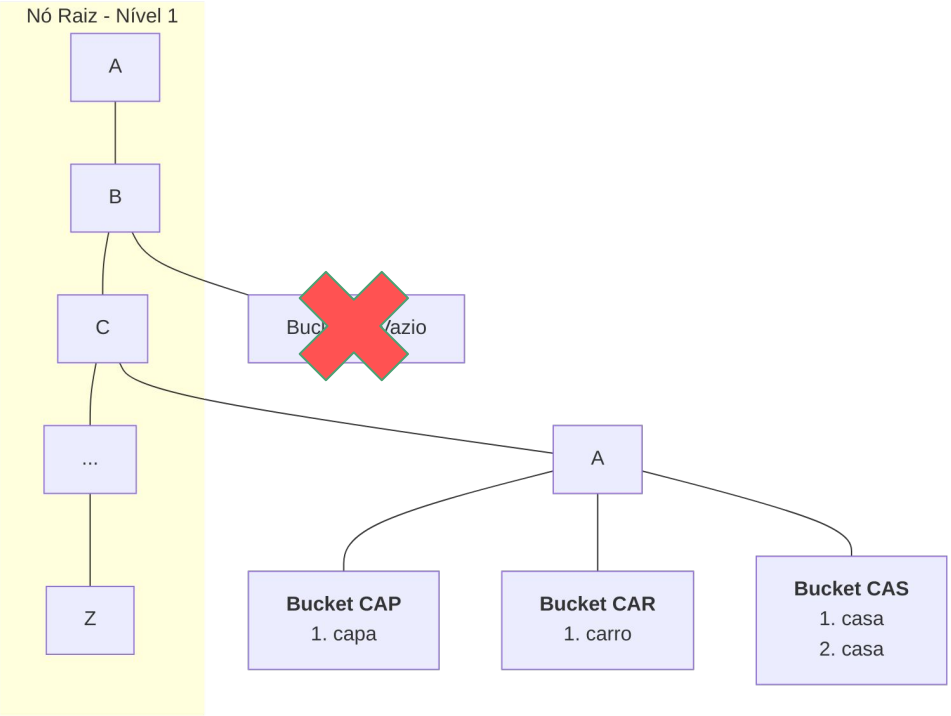


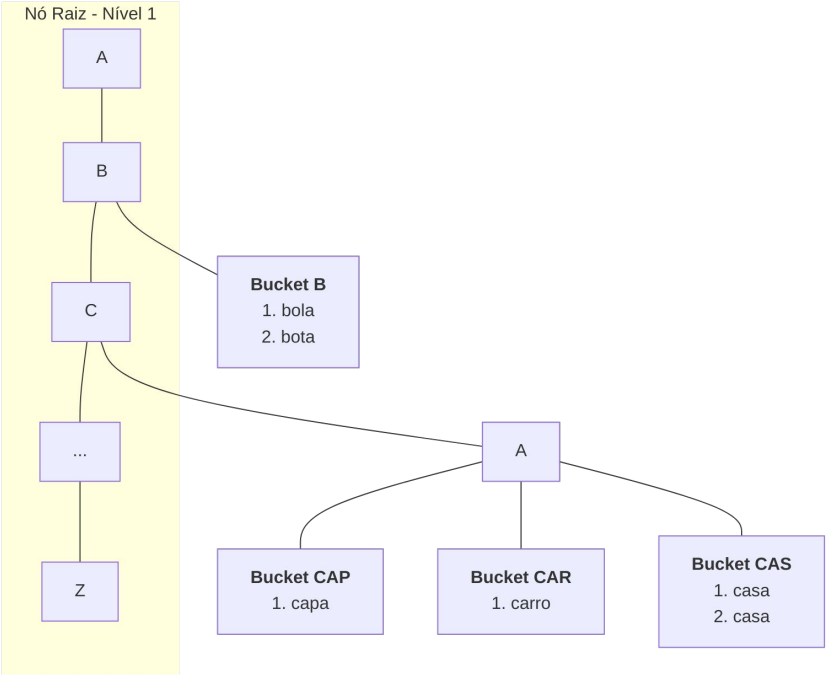




```
["casa", "capa", "carro", "casa", "bola", "bota"]
```

Exemplo





Comparativo de Performance

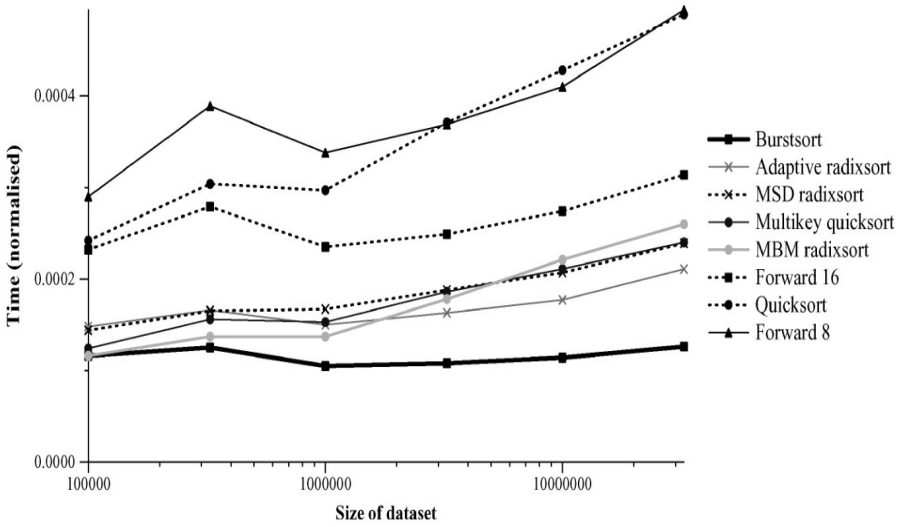


Fig. 3. Duplicates. The vertical scale is time in milliseconds divided by $n \log n$.

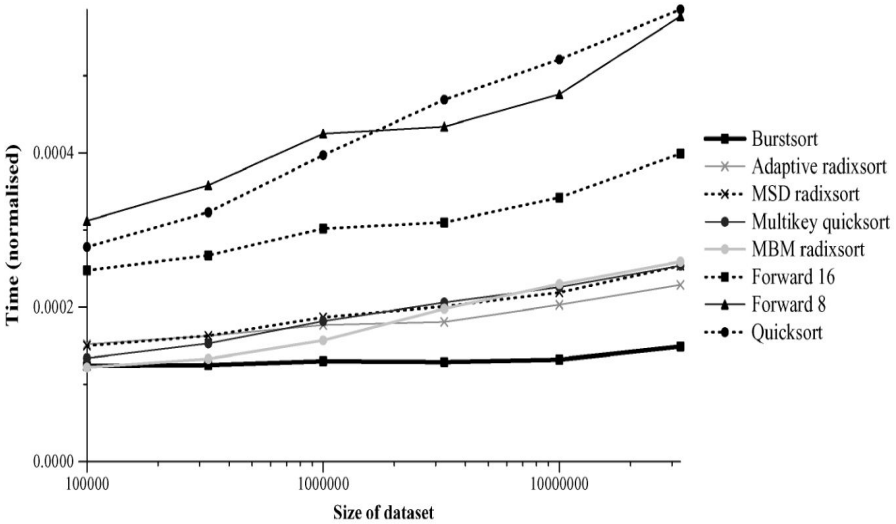


Fig. 4. No duplicates. The vertical scale is time in milliseconds divided by $n \log n$.

Comparativo de Performance

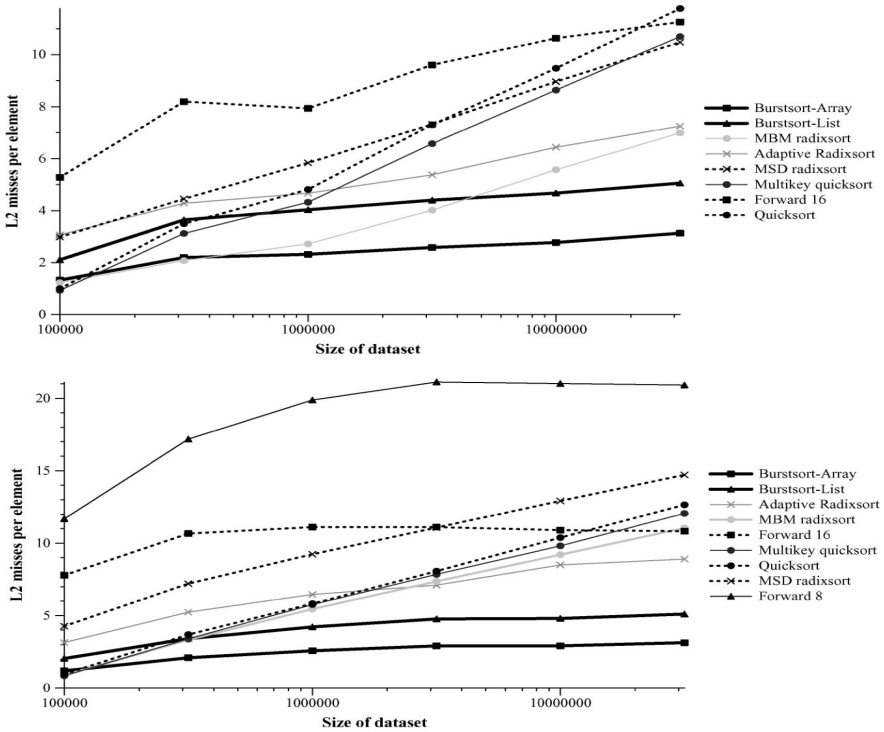


Fig. 9. Cache misses, 1 Mb cache, 8-way associativity, 32 bytes block size. Upper: duplicates. Lower: genome.

Comparativo de Performance

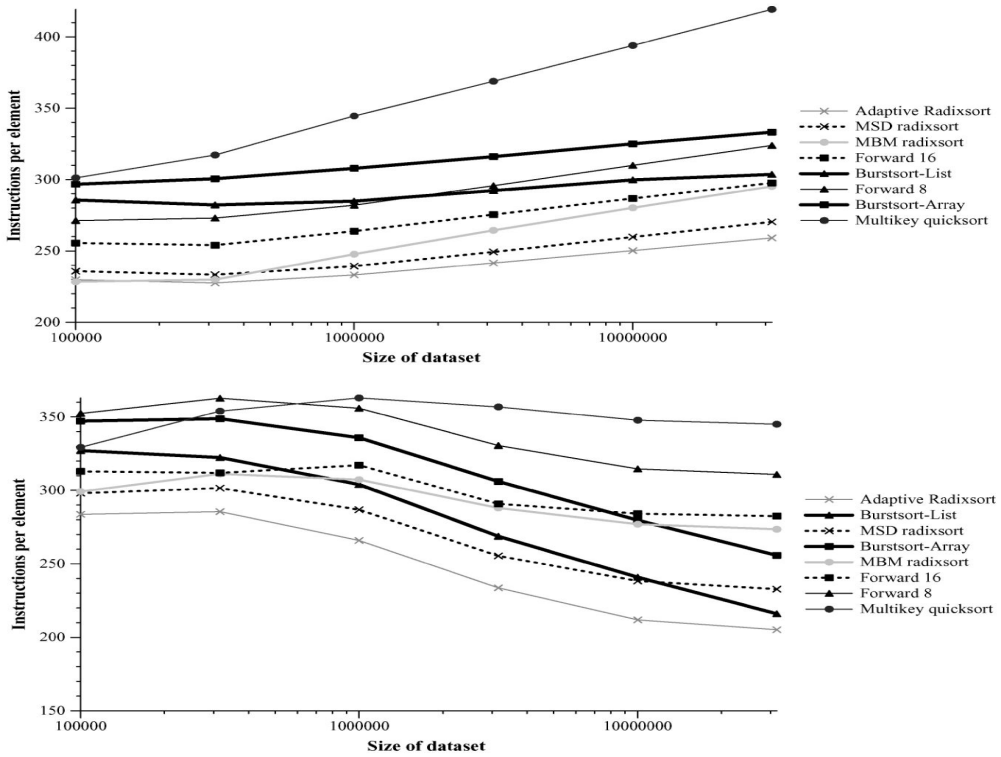


Fig. 11. Instructions per element. Upper: no duplicates. Lower: genome.

Comparativo de Complexidade

n: número de strings.

L: tamanho médio dos buckets (limite para o *burst*).

S: espaço total para armazenar os caracteres das strings.

R: tamanho do alfabeto (Radix).

w: comprimento da maior string (profundidade da recursão).

T: tamanho da estrutura da Trie (nós de acesso e ponteiros).

Dimensão Analítica	**BurstSort**	**MSD RadixSort**	**Multikey Quicksort**
Estratégia	Trie Dinâmica + Buckets	Largura	Profundidade
Tempo (pior caso)	O(n log n)	O(n log n)	O(n^2)
Espaço (pior caso)	$O(n \cdot L+T)$	$O(R \cdot w+S)$	O(w+S)
Padrão de Memória	Localizado (cache-friendly)	Espalhado (cache-hostile)	Moderado
Taxa de Miss L2	Muito baixa	Alta	Média
Estabilidade	*Sim	Sim	Não

Referências

[1] SINHA, R.; ZOBEL, J. **Cache-conscious sorting of large sets of strings with dynamic tries.** *ACM Journal of Experimental Algorithmics*, v. 9, art. 1.5, 2004. DOI: 10.1145/1005813.1041517.

[2] MAKHAN1101. **BurstSort: Implementation of BurstSort algorithm for string sorting.** GitHub, 2023. Disponível em: https://github.com/makhan1101/BurstSort.

[3] WIKIPÉDIA. **Burstsort.** Disponível em: https://pt.wikipedia.org/wiki/Burstsort.